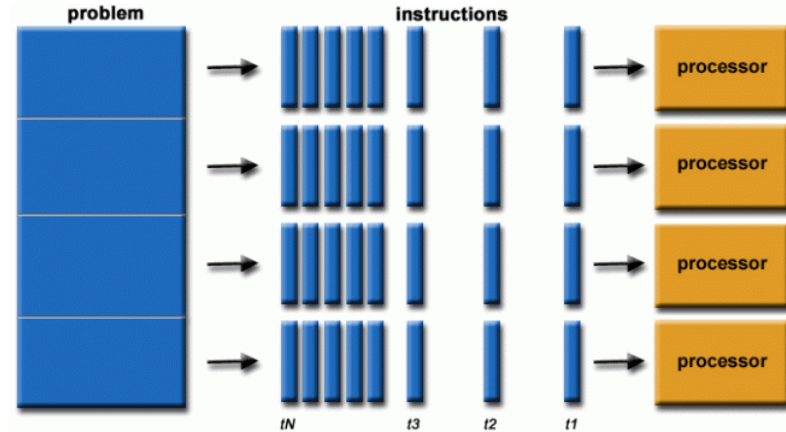
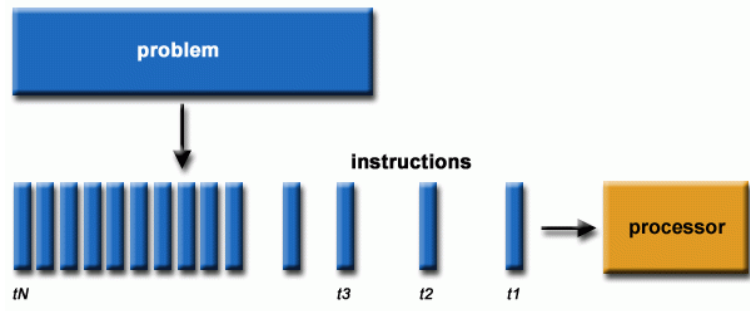


Parallel Programming



- A parallel processing program is a single program that runs on multiple processors simultaneously.
- The overall problem is split into parts, each of which is performed by a separate processor in parallel.
- In addition to faster solution, it may also generate a more precise solution



Terminologies



- . Parallel Processing
 - . Processing multiple tasks simultaneously on multiple processors is called parallel processing
- . Parallel Programming
 - . Software methodology used to implement parallel processing
- . Parallel Computing
 - . Encompasses all the technologies used in running multiple tasks simultaneously on multiple processors



High Performance Computing

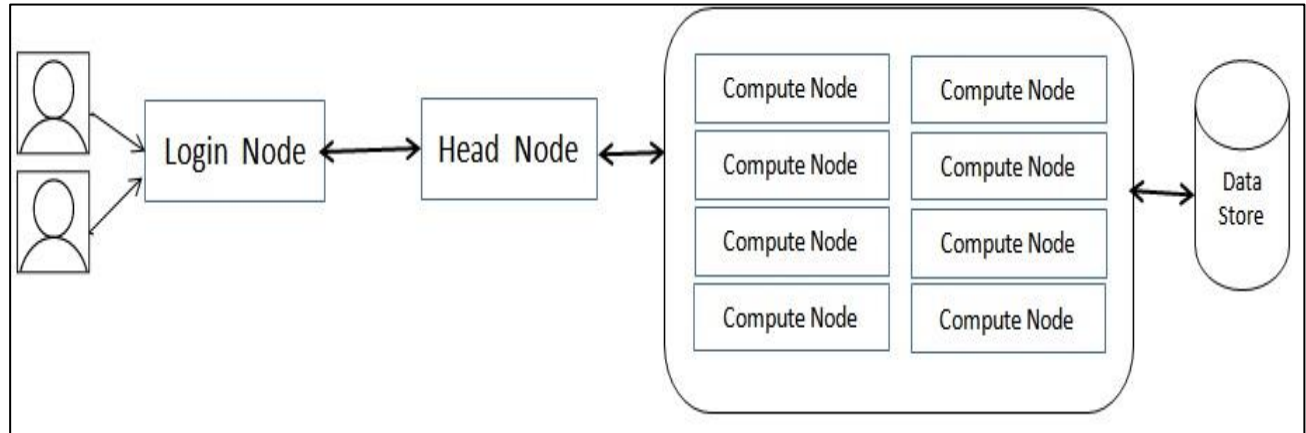


- High performance computing (HPC) is the ability to process data and perform complex calculations at high speeds
- HPC refers to high-speed parallel processing of complex computations on multiple servers. A group of these servers is called a cluster, which consists of hundreds or thousands of computing servers connected through a network
- HPC is a computer with a high-level computational capacity compared to a general purpose computer. Aggregation of computing power Delivers much higher performance than one could get out of a regular desktop

HPC components

Three main components

- Compute
- Network
- Storage





Units of HPC



Weather Forecasting

If 1 cell is of size 1km x 1km x 1km

The whole atmosphere about 5×10^8 cells

- 1 Mflop/s 10^6
- 1 Gflop/s 10^9
- 1 Tflop/s 10^{12}
- 1 Pflop/s 10^{15}
- 1 Eflop/s 10^{18}
- 1 Zflop/s 10^{21}
- 1 Yflop/s 10^{24}

If each calculations require 200 Flops,

- it will take 200 FLOPS/cell * 5×10^8 cells for each time step
- Number of time steps for a 10-day simulation with 10-minute intervals (144 intervals/day) will be 1440
- Total FLOPS = (Number of FLOPS per time step per cell) * (Number of cells) * (Number of time steps)
- Total FLOPS = (200 FLOPS/cell) * (5×10^8 cells) * (1440 time steps)
- Total FLOPS = 1.44×10^{17} FLOPS
- **It would take approximately 1,666 days to compute the simulation with a 100 MFLOPS system and about 20 hours to compute the simulation with a 2 TFLOPS system.**

Performance in FLOPS with scale

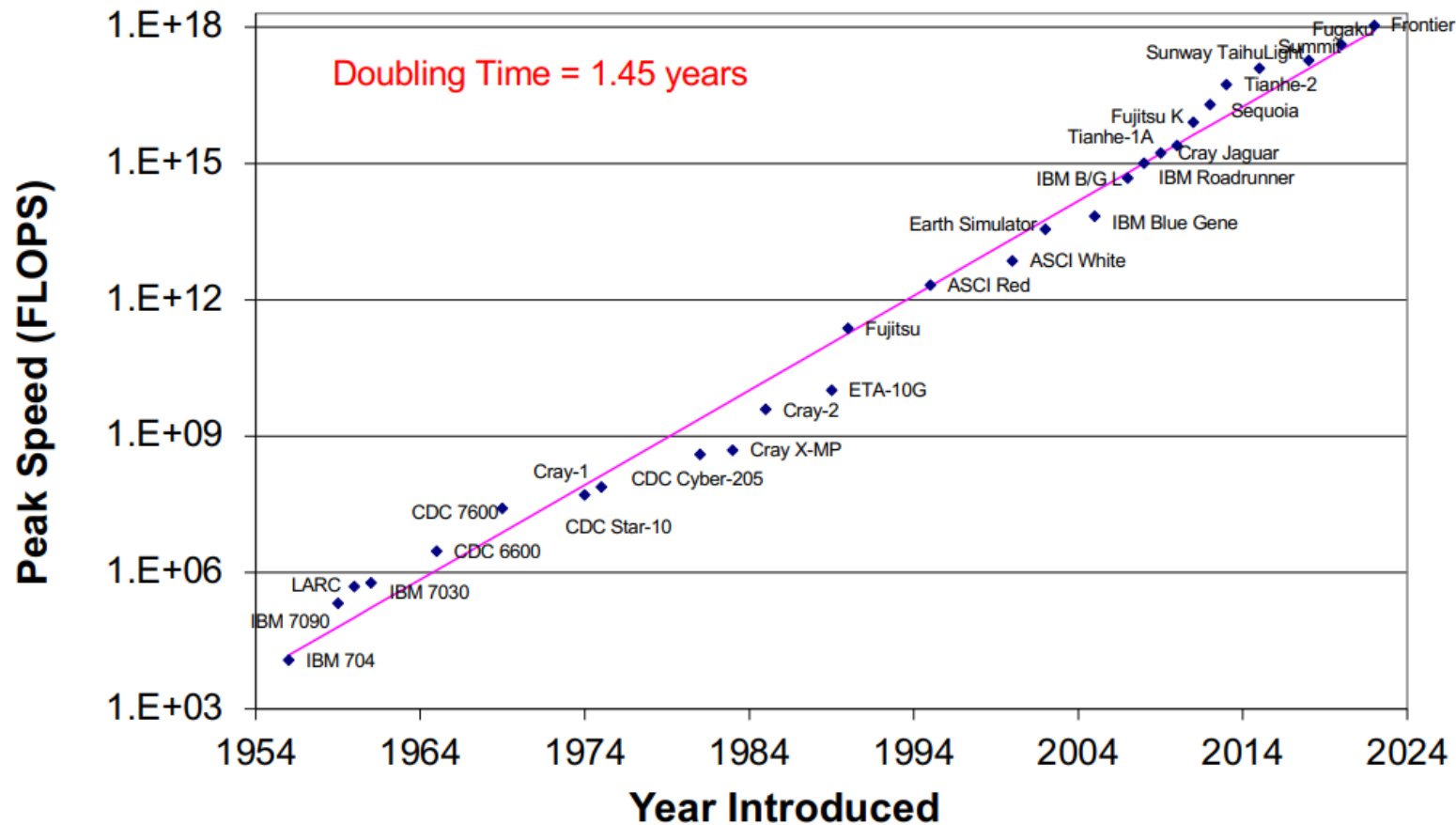
Name	Notation	Exponent	Example
kiloFlops	1,000	10^3	-
megaFlops	1,000,000	10^6	-
gigaFlops	1,000,000,000	10^9	Average Laptop range
teraFlops	1,000,000,000,000	10^{12}	Newest gaming console and graphic cards
petaFlops	1,000,000,000,000,000	10^{15}	Supercomputer range
exaFlops	1,000,000,000,000,000,000	10^{18}	Most Powerful Supercomputer
zettaFlops	1,000,000,000,000,000,000,000	10^{21}	-
yottaFlops	1,000,000,000,000,000,000,000,000	10^{24}	-

Supercomputer

- Supercomputers are a class of high-performance computing (HPC) systems designed to offer higher computational power and processing speed.
- Tightly coupled computers that work together closely as though they are a single computer



Growth of Supercomputer



World Top5 Supercomputers(<https://top500.org/>)

Rank	Site	System	Cores	Rmax (PFlop)	RPeak (PFlop)
1	Frontier , DOE/SC/Oak Ridge National Laboratory United States	HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE	8,699,904	1,194.00	1,679.82
2	Aurora, DOE/SC/Argonne National Laboratory United States	HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel	4,742,808	585.34	1,059.33
3	Eagle, Microsoft Azure United States	Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR	1,123,200	561.20	846.84
4.	A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu	A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu	7,630,848	442.01	537.21
5.	LUMI, EuroHPC/CSC Finland	HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE	2,220,288	309.10	428.70



Terminology



- **FLOPS (Floating Point Operations Per Second)**: FLOPS is a fundamental measure of computational speed, representing the number of floating-point calculations a supercomputer can perform in one second. It is often used to quantify a supercomputer's raw processing power.
- **Top500 List**: The Top500 list is a semiannual ranking of the world's most powerful supercomputers. It considers the LINPACK benchmark results and ranks supercomputers based on their performance. The top-ranked supercomputer is considered the fastest in the world.
- **Rmax**: metric that measures the maximum sustained performance of a supercomputer when running real-world applications and simulations. It is typically expressed in FLOPS.
- **Rpeak**: represents the theoretical peak performance of a supercomputer, indicating the maximum computational speed it can achieve in ideal conditions. This is usually higher than the real-world Rmax performance.

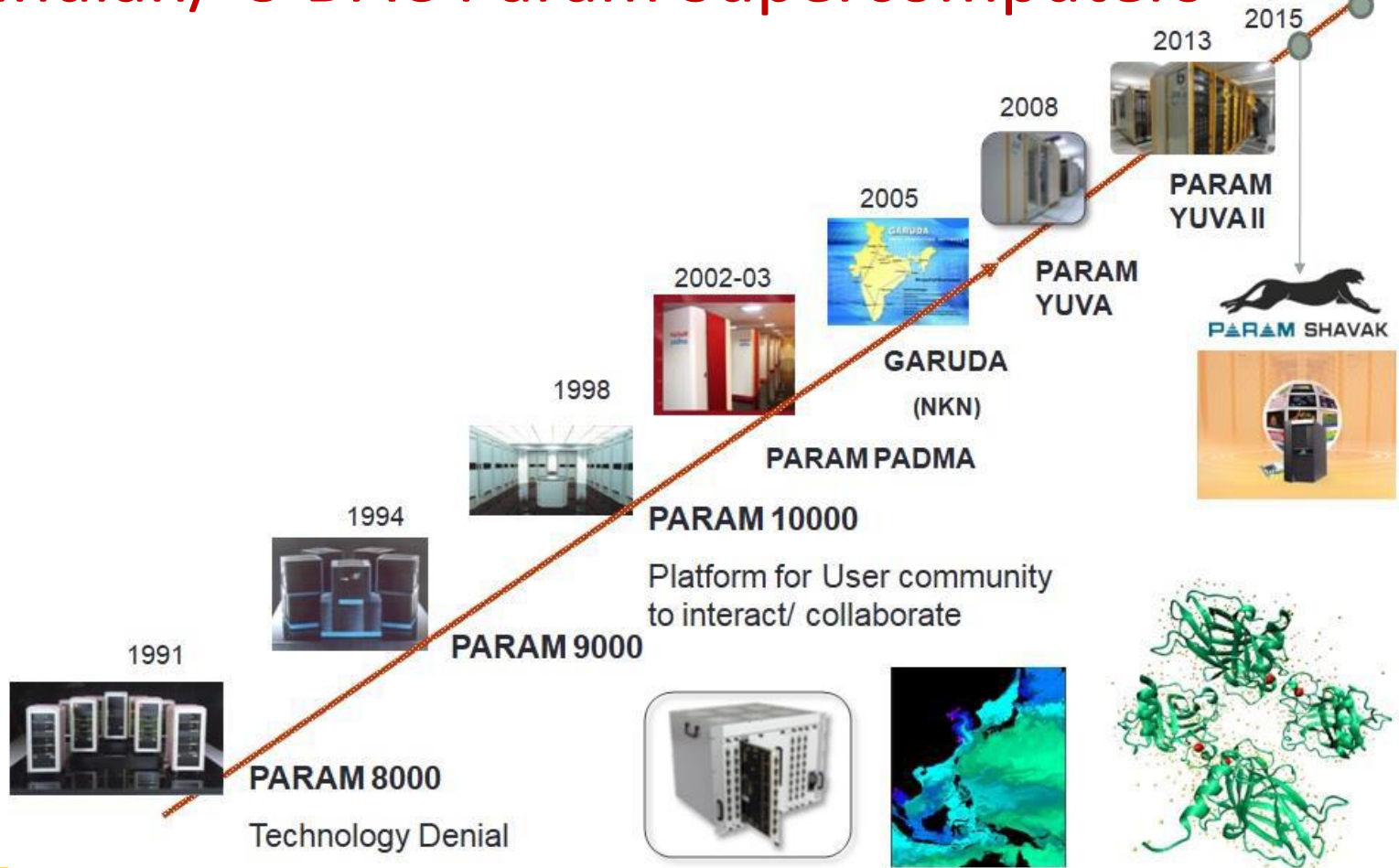


Terminology



- **LINPACK Benchmark:** The LINPACK benchmark is a widely used method to assess a supercomputer's performance. It measures how quickly a system can solve a dense system of linear equations and provides a standardized way to compare different supercomputers.
- **HPL (High-Performance Linpack):** HPL is a variant of the LINPACK benchmark used in the Top500 list. It is a standardized test to evaluate a supercomputer's performance.
- **Energy Efficiency:** Energy efficiency measures how efficiently a supercomputer uses power to perform calculations. It is expressed as FLOPS per watt and helps determine the environmental impact and cost-effectiveness of a supercomputer.
- **Scalability:** Scalability assesses a supercomputer's ability to handle larger workloads by adding more processing nodes or components. Scalability is crucial for accommodating growing computational demands.

Year	Name	Peak speed	Location
1960	LARC	500 kFLOPS (2 CPUs)	Lawrence Livermore Lab. USA
1961	IBM 7030	1.2 MIPS 600 kFLOPS	Los Alamos Lab. USA
1965	CDC 6600	10 MIPS 3 MFLOPS (10^6)	Lawrence Livermore Lab. USA
1965	CDC 7600	36 MFLOPS	Lawrence Livermore Lab. USA
1975	Cray-1	80 MFLOPS (vector) 72 MFLOPS (scalar)	Los Alamos Lab. USA
1985	Cray-2	1.95 GFLOPS (4 CPUs) 3.9 GFLOPS (8 CPUs) (10^9)	Lawrence Livermore Lab. USA
1995	Intel ASCI Red	2.15 TFLOPS (10^{12})	Sandia National Lab. USA
2010	Tianhe-1A	2.57 PFLOPS (10^{15})	National Supercomputer Centre, Tianjin, China
2022	Frontier	1102 PFLOPS (10^{18})	Oak Ridge National Lab, USA





Indian Top5 Supercomputers

(<https://topsc.cdacb.in>)

Rank	Site	System	Cores/ nodes	Rmax (TFlop)	RPeak (TFlop)
1 [90]	Airawat-PSAI, C-DAC Pune	NVIDIA DGX A100, AMD EPYC 7742 64C 2.25GHZ, NVIDIA A100, INFINIBAND HDR2	81344	8500	13176
2 [163]	PARAM Siddhi-AI, C-DAC,Pune	NVIDIA DGX A100, AMD EPYC 7742 64C 2.25GHZ, NVIDIA A100, Mellanox HDR Infiniband (OEM:ATOS under NSM initiative, Bidder)	41664/ 236	4619	5267.14
3 [201]	Pratyush, Indian Institute of Tropical Meteorology, Pune	Cray XC-40 class system with 3315 CPU-only (Intel Xeon Broadwell E5-2695 v4 CPU) nodes with Cray Linux environment as OS, and connected by Cray Aries interconnect	119232/ --	3763.9	4006.19
4. [354]	Mihir, NCMRWF, Noida	Intel Xeon Broadwell E5-2695 v4 CPU with Cray Linux environment connected by Cray Aries interconnect OEM:Cray, Bidder:Cray	83592/ 1152	2570.4	2808.7
5.	PARAM Pravega, IISc, Bangalore	Intel Xeon Cascade Lake processors,NVIDIA Tesla V100 with NVLink, Mellanox HDR interconnect. OEM:Atos, Bidder:Atos	29952/ 624	1702	2565



NATIONAL SUPERCOMPUTING MISSION

INFRASTRUCTURE | APPLICATIONS | R&D | HRD



- Jointly Steered by Department of Science and Technology (DST) and Ministry of Electronics and Information Technology (MeitY)

Mission Statement:

- The Mission envisages empowering our national academic and R&D institutions spread over the country by installing a vast supercomputing grid comprising of more than 70 high-performance computing facilities.

Objective:

- To make India one of the world leaders in Supercomputing and to enhance India's capability in solving grand challenge problems of national and global relevance
- To empower our scientists and researchers with state-of-the-art supercomputing facilities and enable them to carry out cutting-edge research in their respective domains
- To minimize redundancies and duplication of efforts, and optimize investments in supercomputing
- To attain global competitiveness and ensure self-reliance in the strategic area of supercomputing technology

NSM Sites

- PARAM Shivay, IIT BHU, 837 TFLOPS
- PARAM Shakti, IIT Kharagpur, 1.66 PFLOPS
- PARAM Brahma, IISER Pune, 797 TFLOPS
- PARAM Yukti, JNCASR Bengaluru, 838 TFLOPS
- PARAM Sanganak, IIT Kanpur, 1.67 PFLOPS
- PARAM Pravega, IISc Bangalore, 3.3 PFLOPS
- PARAM Seva, IIT Hyderabad, 838 TFLOPS
- PARAM Smriti, NABI, Mohali, 838 TFLOPS
- PARAM Utkarsh, C-DAC Bengaluru, 838 TFLOPS
- PARAM Siddhi-AI, C-DAC Pune,
- PARAM Ganga, IIT Roorkee, 1.67 PFLOPS
- PARAM Ananta, IIT Gandhinagar, 838 TFLOPS
- PARAM Porul, NIT Trichy, 838 TFLOPS
- PARAM Himalaya, IIT Mandi, 838 TFLOPS
- PARAM Kamrupa, IIT Guwahati, 838 TFLOPS

PARAM Shivay



PARAM Shakti



PARAM Brahma



PARAM Yukti



PARAM Sanganak



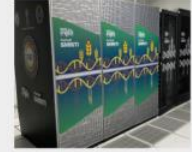
PARAM Pravega



PARAM Seva



PARAM Smriti



PARAM Utkarsh



PARAM Ganga



PARAM Ananta



PARAM Porul



PARAM Himalaya



PARAM Kamrupa





Param Utkarsh

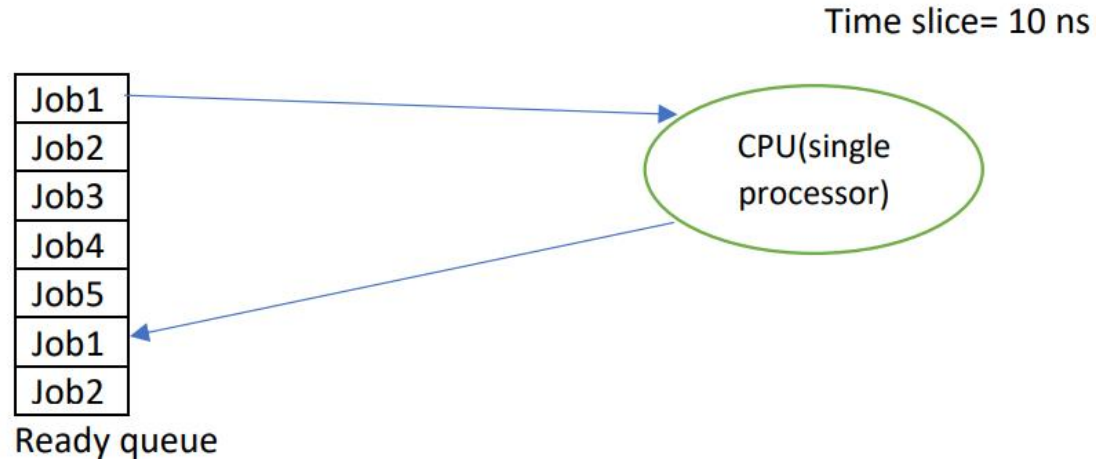
परम
उत्कर्ष
PARAM
Utkarsh



Parallel Programming Concepts

Concurrency

- Concurrency is the management of multiple tasks at the same time
 - e.g. Sharing a CPU across multiple processes
- Concurrency means, In no particular order, multiple tasks that start, run, and complete in overlapping time periods.



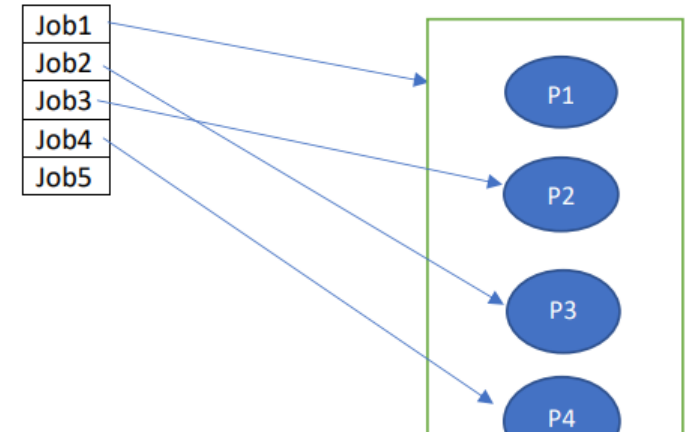
When we speak about a minimum of two tasks or more, concurrency is basically valid.

If an application is able to virtually perform two tasks at the same time, we call it a concurrent application.

Even though tasks run here appear to be simultaneous, but basically they Are not.

They take advantage of the operating system 's Processor time-slicing feature, where each task runs part of its task and then goes to the waiting state. When the first task is in a waiting state, the CPU is assigned to the second task to complete part of its task.

- Parallelism is using multiple resources to perform multiple tasks at the same time
 - e.g. multiple cores for tasks, vector instructions
- Parallelism is when, for example, on a multi-core processor, several tasks OR several parts of a unique task actually operate at the same time.
- Concurrency and parallelism are NOT the same thing.





Parallelism vs Concurrency



- Parallel program uses parallel hardware to execute computations more quickly. Efficiency is its main concern. as such parallel programming is mainly concerned with efficiency.
- Parallel programming answers questions such as how to divide a computational problem into sub problems that can be executed in parallel.
- How to make best use of underlying parallel hardware to obtain the optimal speed up.
- eg - matrix multiplication, data processing, computer graphics rendering or simulation of fluid movement

- Concurrent programming serves to structure programs better in order to improve modularity, responsiveness to input and output events or the understandability of the program.
- concerned with questions like when can a specific execution start
- when and how can 2 concurrent executions exchange information.
- Targets writing asynchronous applications such as web servers, user interfaces or databases
- Concerned with convenience, better responsiveness and maintainability



Card Sorting Example



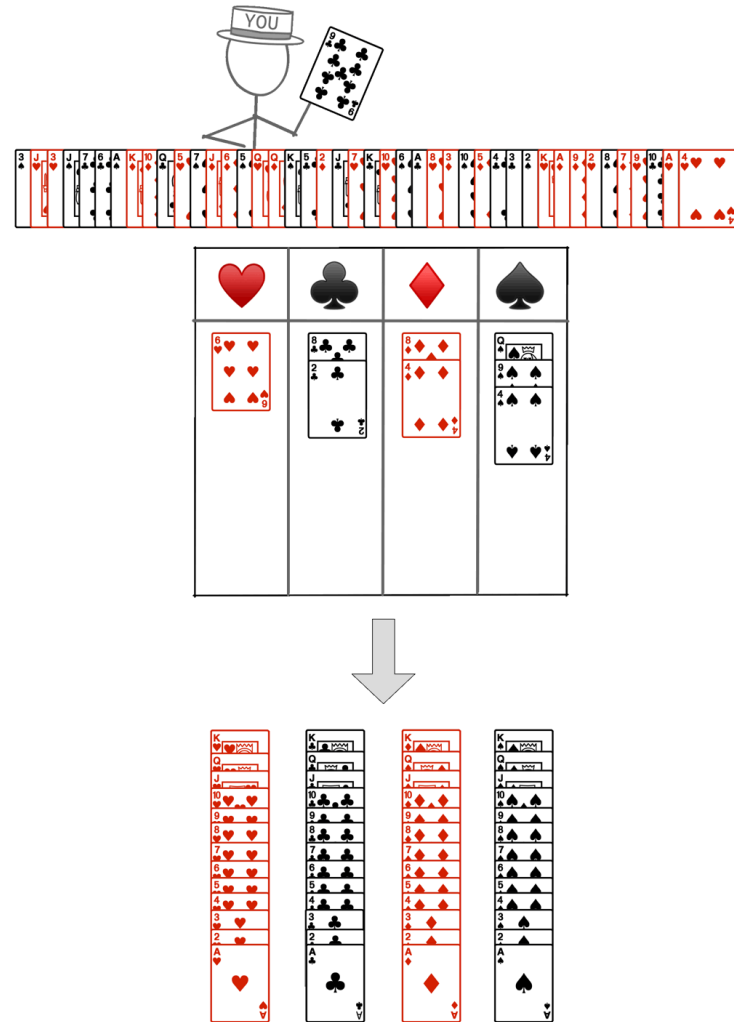
Friends working together to sort a deck of playing cards. We will consider a few ways people might work together to sort cards and use these examples to introduce fundamental parallel computing concepts.

Observe the following:

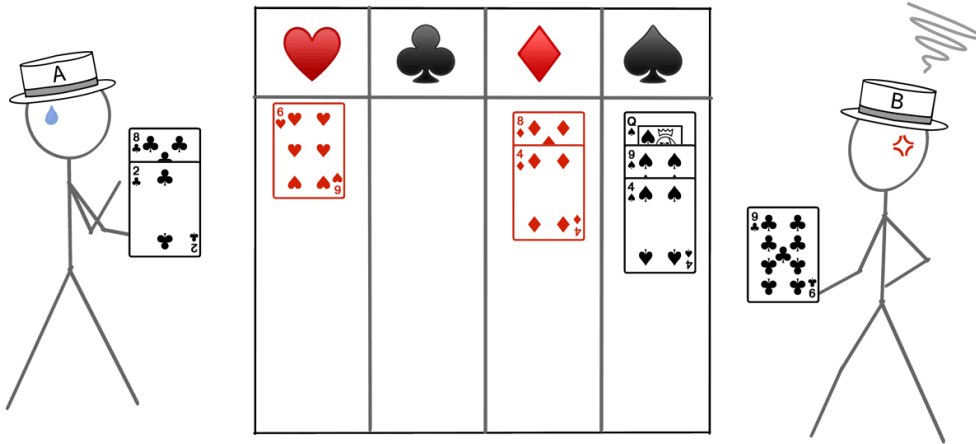
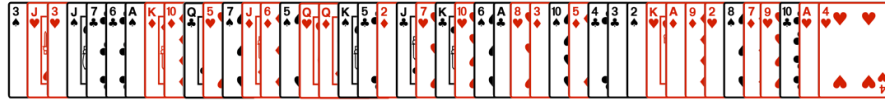
- Are some tasks more efficient to do by yourself?
- Does adding additional helpers always speed up a task?
- Is there a limit to how many helpers can work together?
- How can people resolve coordination issues while they work together?
- Does the way the task and helpers are organized impact how well they can work together?

One Worker - approach

- Sort a deck of cards by color, suit, and face value.
- designate a pile for each suit and insert cards into the piles in sorted order?
- card sorting strategy (*algorithm*) need to make many comparisons (*operations*), one at a time.
- serial program*, executing a sequence of operations, one at a time.
- No matter how clever your sort algorithm, you will eventually reach a limit on how quickly you can perform this sorting task by yourself.



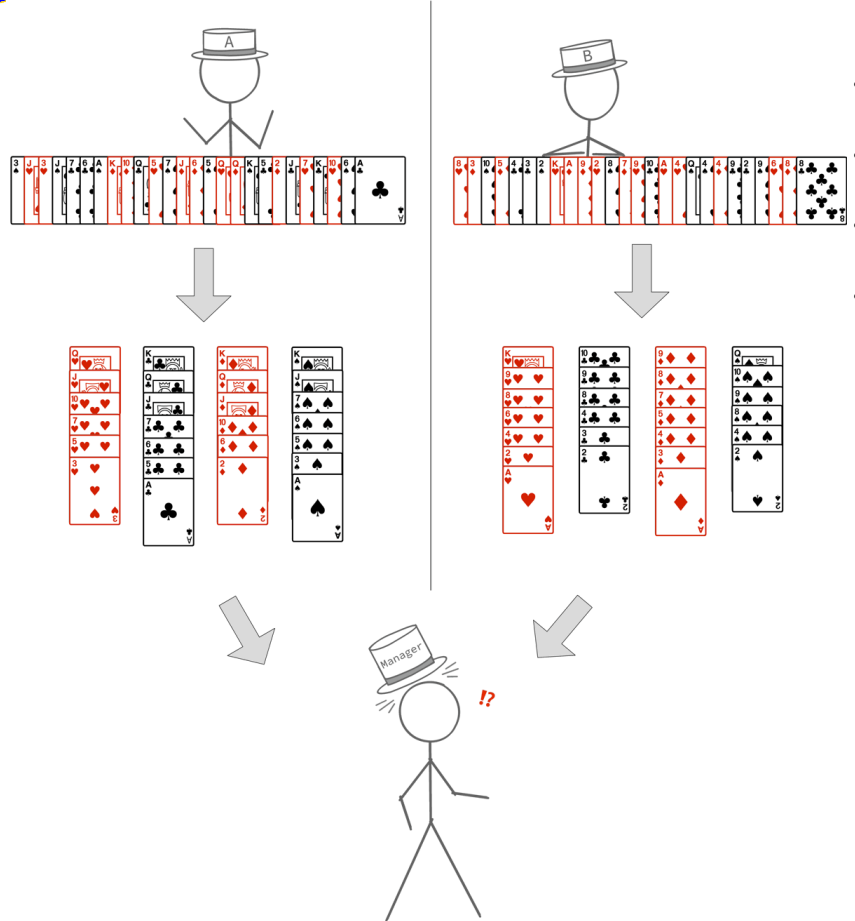
Working Together – 2 in parallel



Two workers need to insert cards from the same suit. Worker A is holding the partial results for the clubs suit, so Worker B is temporarily blocked.

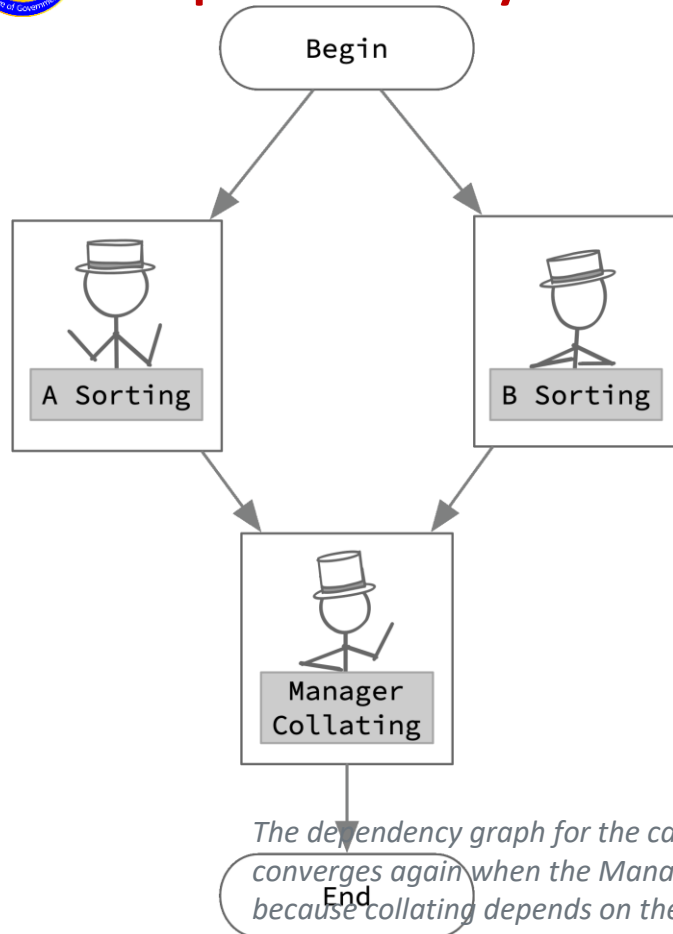
- Work from one pile of unsorted cards.
- all workers have a view of current progress and activity.
- they produce one set of sorted cards.
- they use the same strategy as the single worker.
- workers are working at the same time (*in parallel*).
- A workflow where parallel workers share a workspace is called **shared memory parallelism**
- Advantage – quickly, speed
- Limitation ??
- Max workers, Avg workers
- Resource Contention

Working Independently- distributed



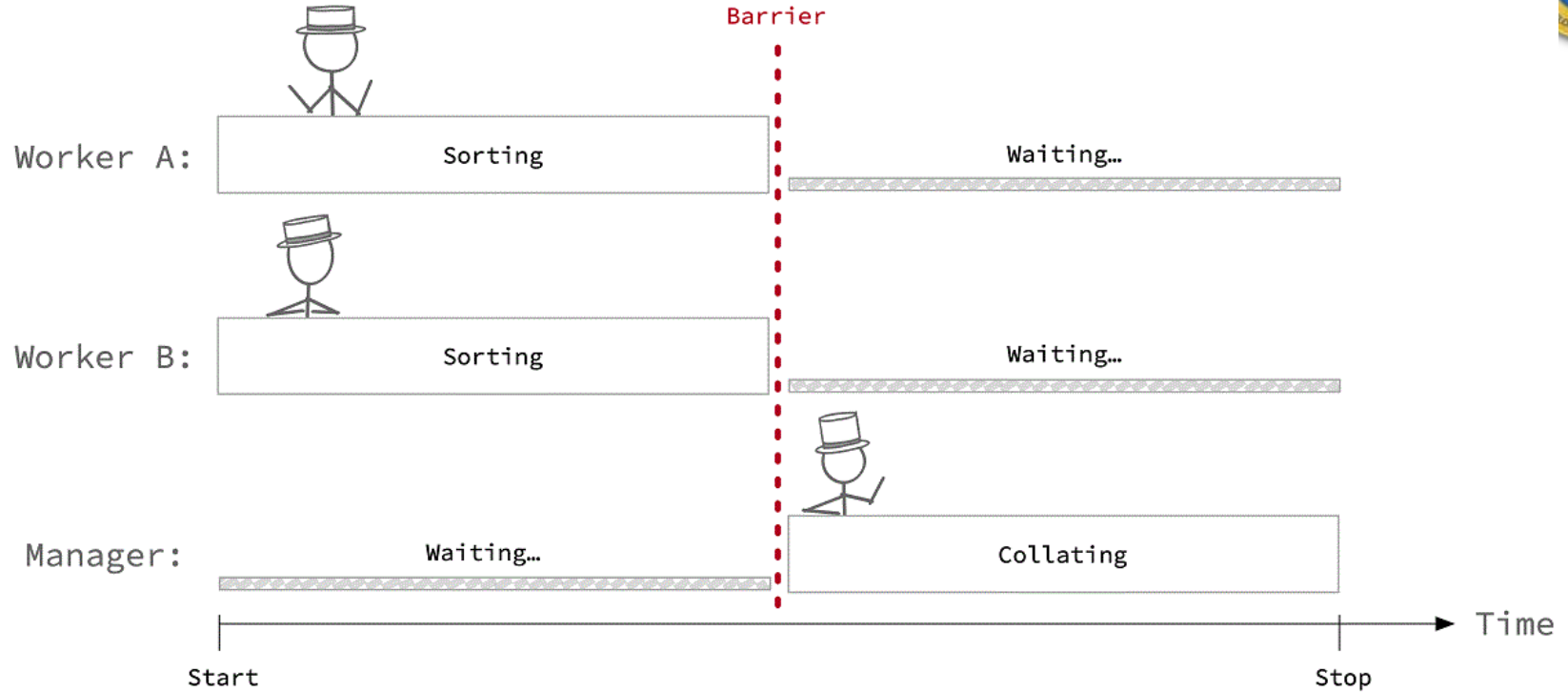
- Solves the resource contention but..
- One result per worker
- Extra step – combining these output
- The amount of work required to coordinate multiple workers can increase rapidly with the number of workers.

Dependency

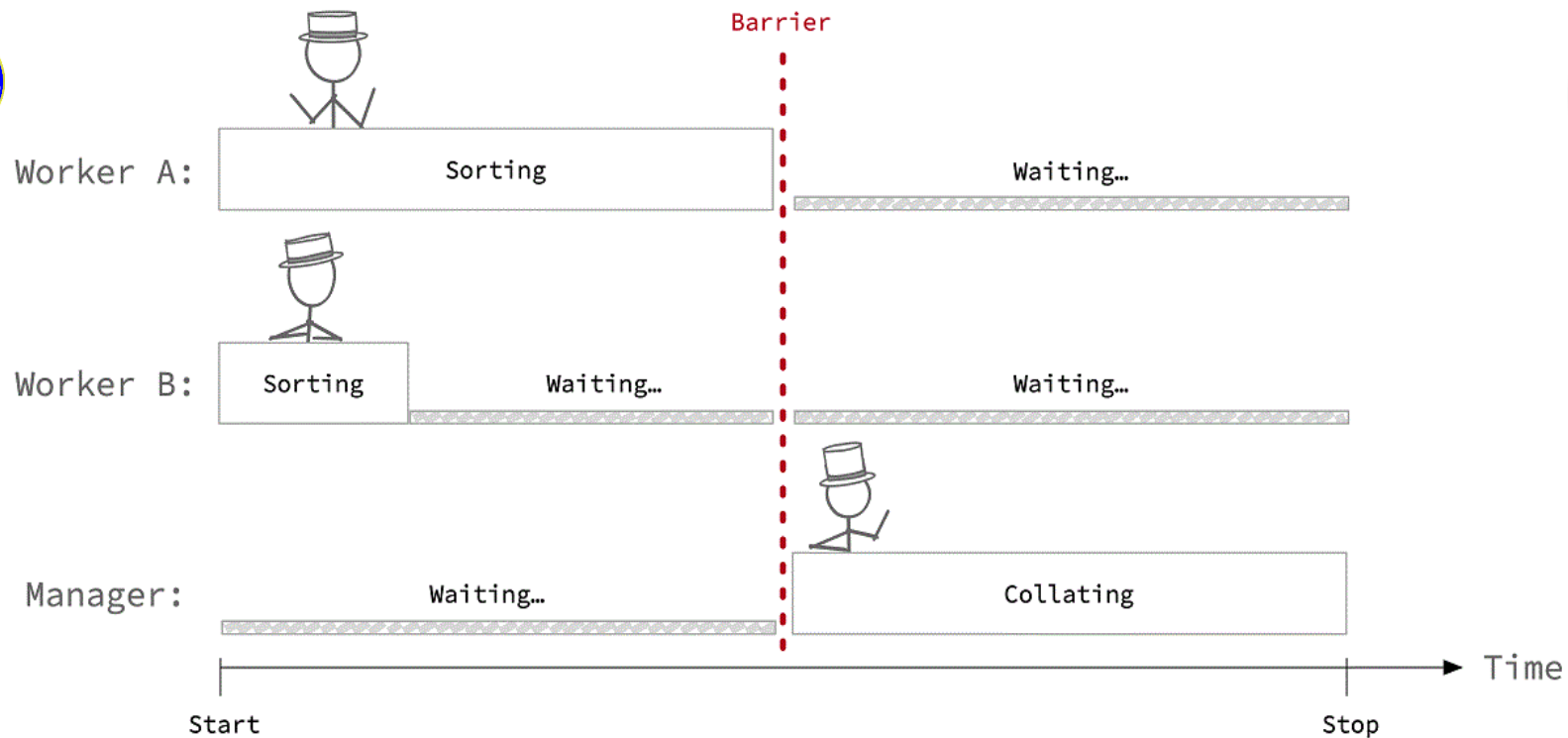


The dependency graph for the card sorting example branches when Worker A and B are sorting in parallel and converges again when the Manager collates the output. The collating task cannot be done in parallel with the sorting task, because collating depends on the output of the sorting steps.

- Sorting Workflows has different stages, which turn input to a desired output.
- Workers in one stage depend on the output from a prior stage.
- When one worker depends on another's output, it is called a *dependency*.
- Consider all the dependency relationships for all the workers in workflow — we can visualize that dependency structure for the workflow as a network (*graph*).
- A group of workers that do not depend on each other's output can work at the same time, in parallel.
- Parallel work splits the dependency graph into multiple branches.



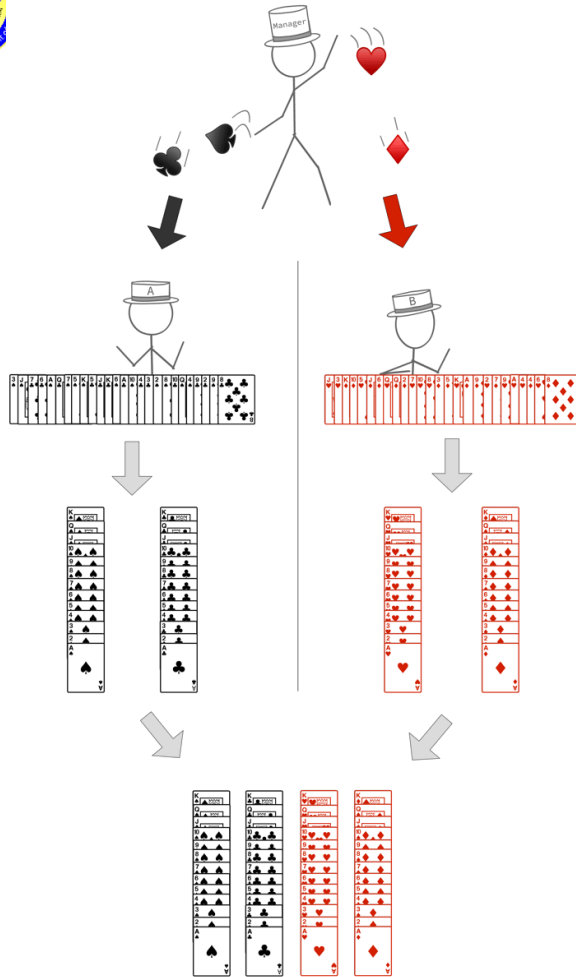
- In our example, the workflow starts with two active workers sorting cards. When both workers have finished sorting, the manager begins collating the results.



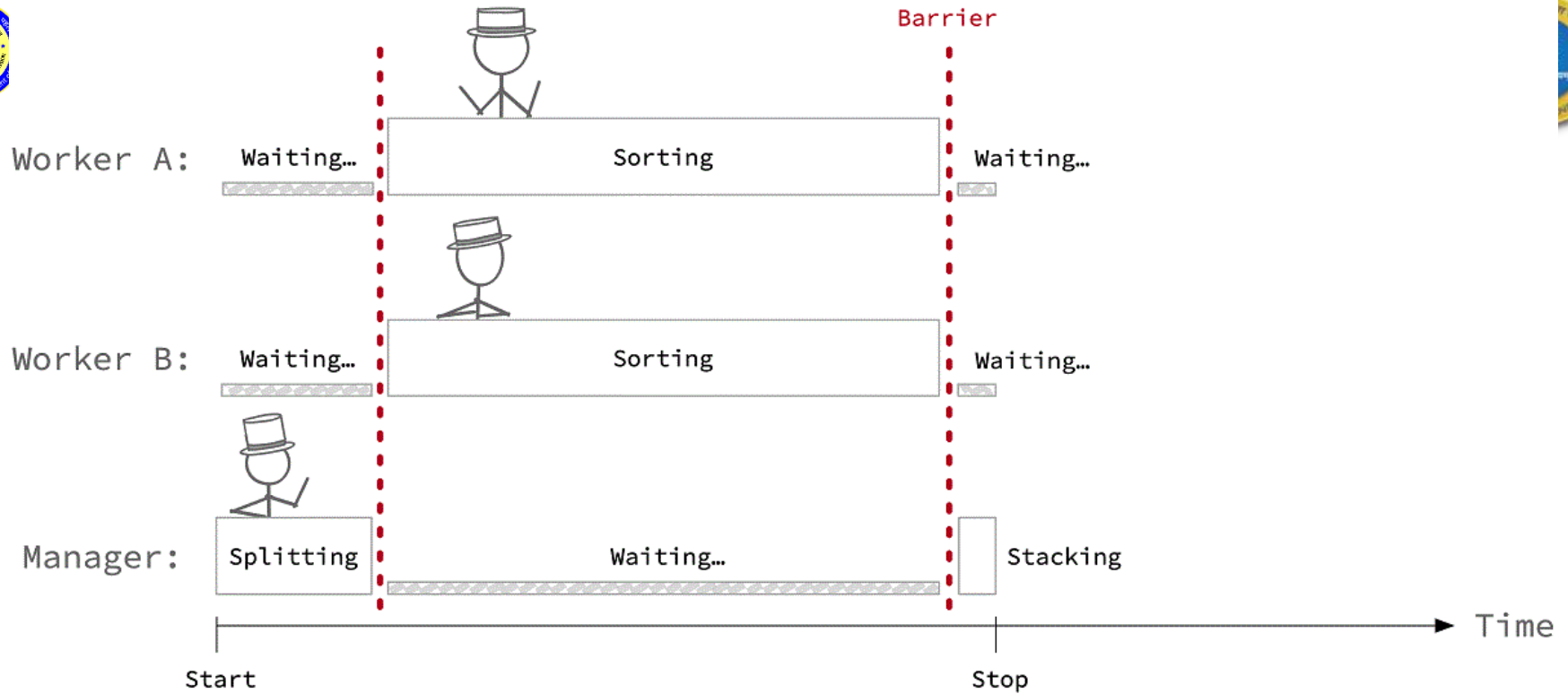
- What if Worker B was exceptionally fast? Could this speed up our workflow? Unfortunately, the manager's task is dependent on the output of all workers, so worker B (and the manager) end up waiting for the slowest worker.
- The collating stage cannot proceed until every worker finishes sorting, so the workflow, as a whole, is not faster. When you are writing parallel programs, it is important to distribute the work to minimize idle time. Suppose the work time per task is highly variable. In that case, you might use a manager to dynamically re-distribute the work among workers to keep them all busy.



Designing for parallel workers



- Split the cards by color and assign each worker to sort one color.
- The workers would follow the same card sorting strategy, producing sorted stacks of same suit cards.
- The early partitioning strategy means that the workers' outputs are complete, sorted, same-suit piles.
- At the end of the workflow, no extra effort is required to combine the workers' outputs into a final result.



- While it does not make much difference with two workers, pre-partitioning the data saves substantial time as the number of workers increases (in this example). It would not take any extra time for the manager to split the cards by suit among four workers than to split the cards by color among two workers. The four workers would each have fewer cards to sort, and the results are just as easy to combine. When designing a parallel workflow, consider how the data you assign to your workers impacts their efficiency and the usefulness of their output.



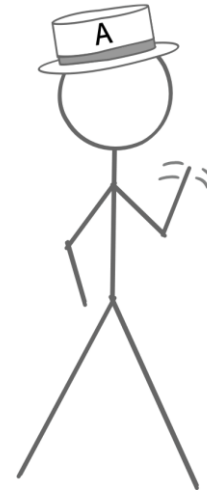
Card sorting examples illustrate parallel concepts



- Increasing the number of parallel workers can decrease the amount of time needed to reach a solution or allow you to tackle larger problems.
- There is a **limit on the number of parallel workers** that can share a resource. The contention for a resource increases as more workers try to use it.
- Independent workers that share no resources still need some amount of coordination **to produce coherent results (communication overhead)**.
- After a point, additional workers do not speed up a workflow. In fact, the **communication overhead and resource contention** can make the overall workflow slower.
- Parallel workflows need to account for the **extra work required to coordinate parallel workers**.
- Thoughtful design reduces communication overhead and is more amenable to scaling up to larger problems.

Threads and Process

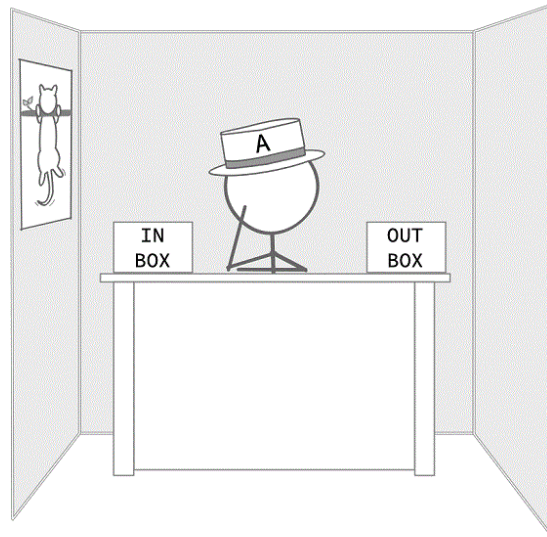
- A *thread* is a sequence of CPU instructions that the operating system treats as a single unit for scheduling and execution purposes.
- Each thread must be associated with a process that manages the resource allocation necessary for the thread's execution.



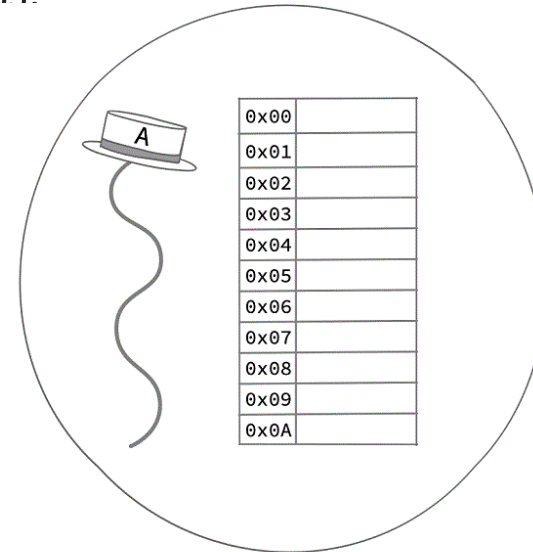
card sorting worker is like a thread in a parallel program

a **process** is a coherent unit of resource allocation, including memory, file handlers, ports, and devices.

- A single process may manage the resources for several threads. Because of the one-to-one or one-to-many relationship between processes and threads, threads can be modeled as components of a process.



A cubicle with one worker.



A process with one thread.

A cubicle, its amenities, and the workers within are analogous to a computer process. A process is associated with one or more threads and a memory allocation

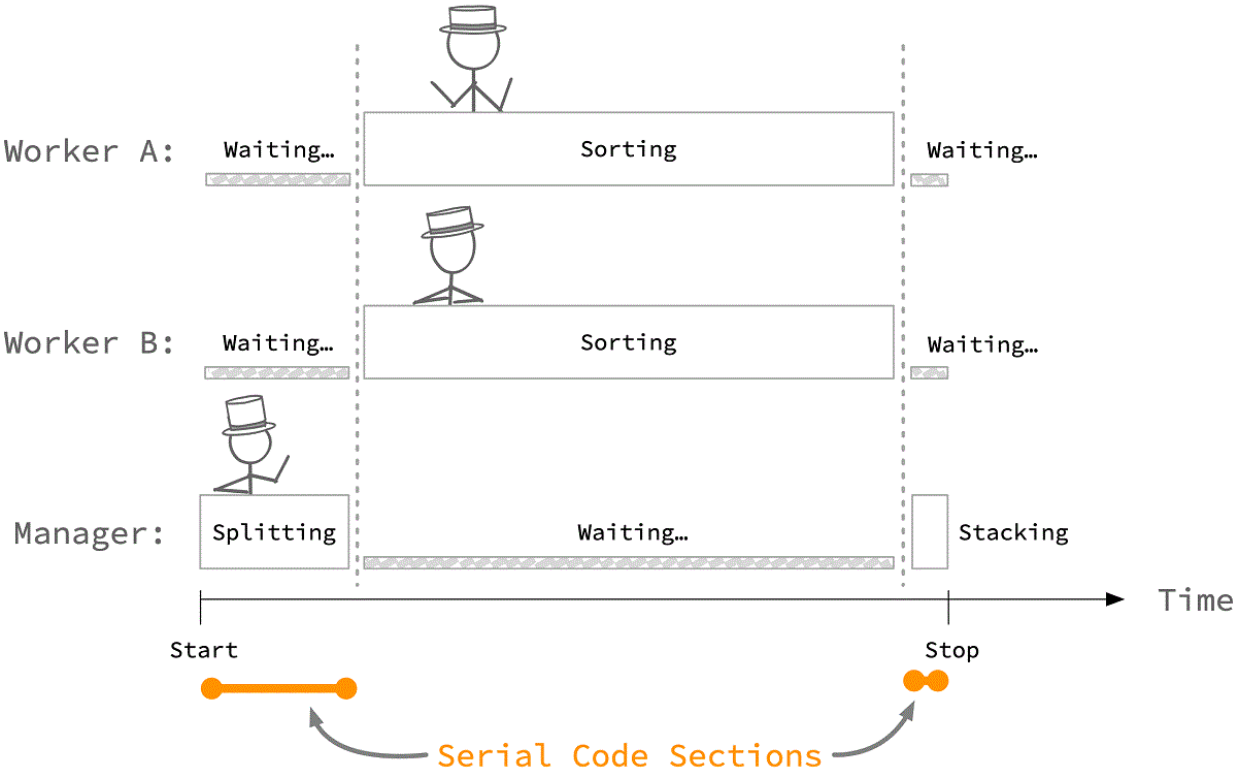


Serial and Parallel Code



- **Serial code** uses a single thread of execution working on a single data item at any one time. Sorting cards using one worker is an example of serial computation. Some parts of a parallel program may still be serial code. For instance, when the manager was partitioning the data by card suit, this was like a serial code section because none of the workers could start until the partitioning was done.
- **Parallel code** has more than one thing happening at a time. In our card sorting example, this corresponds to having multiple workers active at once. In computing this could be:
 - **Multiple threads** of execution **in a single process** (Multiple card sorters working at one table).
 - **Multiple processes** all working **on the same problem** (Multiple workers, each at their own table).
 - **A single thread** of execution **operating on multiple data items** simultaneously (e.g., by issuing vector instructions on a CPU).
 - Any combination of the above.

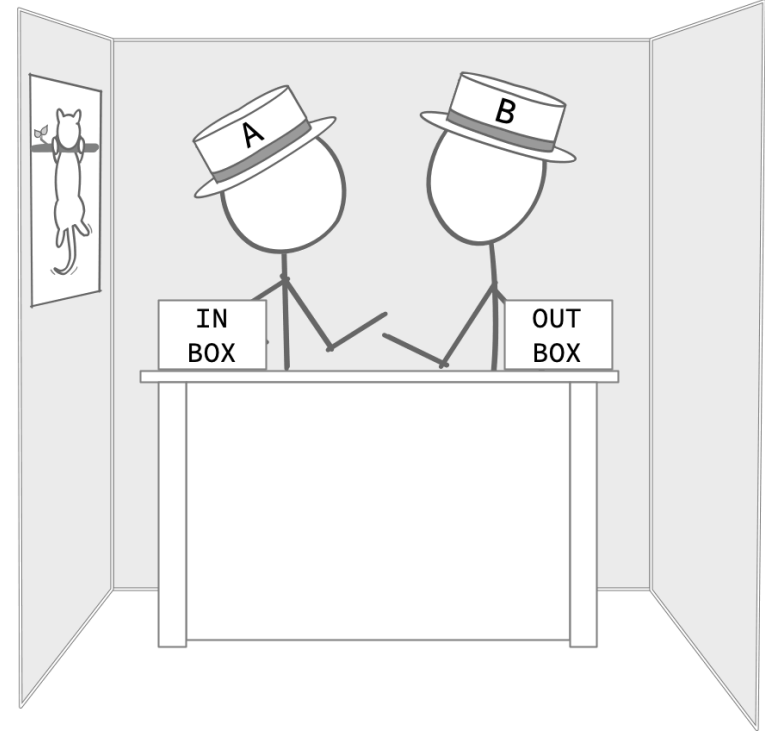
Parallel Code Section



A parallel program is likely to contain some serial sections and some parallel sections.

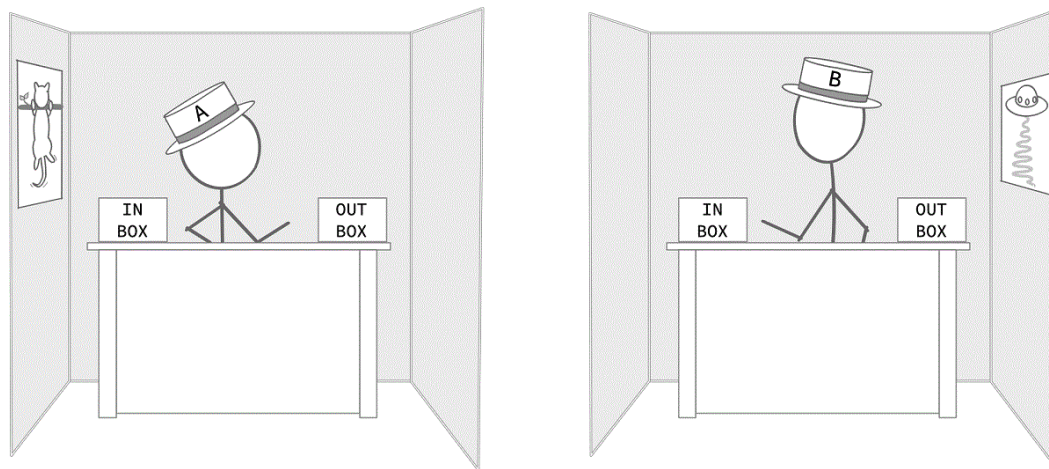
Tasks

- In the context of HPC, a task is any distinct stream of instructions issued by parallel code.
- Depending on the parallelization strategy, a task can correspond to a single thread or a process.
- Conceptually, a task represents both a unit of work and the worker itself
- tasks are like parallel workers that perform computation and others as managers that coordinate input and results. Our card sorting example used both managers and workers.
- Tasks correspond to threads running within a single shared process. Because these tasks share the same process, they can communicate using the shared memory allocated to the process. The OpenMP API facilitates this kind of shared memory parallelism.



In a shared memory design, multiple workers occupy same cubicle (*process*) simultaneously.

- **Tasks can also be different processes**, such that the **available memory is distributed** across tasks rather than shared. Because they do not rely on shared memory, these tasks can run on different computers, allowing the parallel workflow to use more memory and CPU cores. Although it is more complicated, distributed memory parallelism is more scalable than shared memory parallelism
- **MPI is a common interface** standard to facilitate communication among tasks that do not share memory. An MPI task corresponds to a process and the associated main thread. Each MPI task is identifiable via a unique numerical ID, called a rank. If an MPI task creates additional threads within the process — using OpenMP or some other mechanism — these threads can be thought of as subtasks.



In a distributed memory design, parallel workers are assigned to different cubicles (*processes*).



Observations



- Goal a parallel program is to create various tasks that execute independent computations while coordinating communication among the tasks.
- The program might do this using MPI, OpenMP, or some other means.
- It can even use a combination of these, e.g., MPI tasks may create subtasks using OpenMP.
- In our cubicles and workers analogy, using MPI and OpenMP together would create multiple cubicles with multiple workers in each cubicle.



Levels of Parallelism



Parallelism is a technique that allows computer hardware to perform multiple tasks simultaneously, increasing the speed and efficiency of processing

There are different types of parallelism for computer hardware, depending on the level and scope of the tasks

- Bit Level Parallelism
- Instruction Level parallelism
- Data Level parallelism
- Task Level parallelism



Bit Level Parallelism



- It is the form of parallel computing which is based on the increasing processor's size.
- It reduces the number of instructions that the system must execute in order to perform a task on large-sized data.
- Example: 8-bit processor must compute the sum of two 16-bit integers.
- It must first sum up the 8 lower-order bits, then add the 8 higher-order bits, thus requiring two instructions to perform the operation.
- A 16-bit processor can perform the operation with just one instruction



Instruction Level parallelism



- It is the ability of a processor to execute **multiple instructions at the same time** within a single cycle.
- ILP can be achieved by using pipelines, which divide an instruction into smaller stages and process them in parallel.
- For example, a processor can fetch, decode, execute, and write back different instructions in different pipeline stages.
- ILP can also be enhanced by using superscalar architectures, which allow multiple instructions to be issued and executed in parallel within a cycle.



Data Level parallelism



- DLP is the ability of a processor to **operate on multiple data elements** at the same time using the same instruction.
- DLP can be achieved by using vector processors, which have special registers and instructions that can handle vectors of data in parallel.
- For example, a vector processor can add two vectors of numbers in one operation, instead of adding each pair of numbers separately.
- DLP can also be achieved by using single instruction multiple data (SIMD) architectures, which use multiple processing units that can execute the same instruction on different data in parallel.

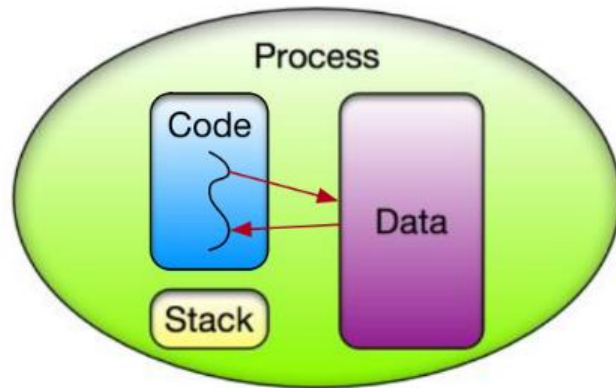


Task Level parallelism

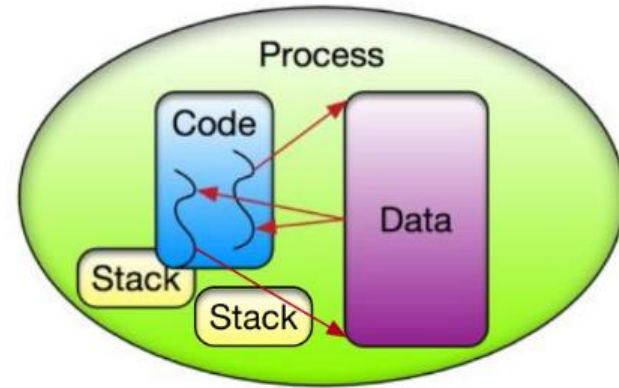


- TLP is the ability of a processor to execute multiple independent tasks or threads at the same time.
- TLP can be achieved by using multiprocessors, which have multiple cores or processors that can run different tasks concurrently.
- For example, a multiprocessor can run multiple applications or processes on different cores or processors.
- TLP can also be achieved by using multithreading, which allows a single core or processor to switch between different threads within a task or application.

Processes and Thread



A process with one thread.



A process with two threads



Critical Sections



- Running multiple threads a single process does not cause problem
- Problem arises when threads access **Shared Memory** – Variables, Arrays or Objects
- A critical section **protects access to shared data** by allowing only one thread to execute the code within it at any point of time
- Concurrent access to shared memory **without synchronization** leads to erroneous behavior
- Critical sections can be protected by **Atomic executions**
- Atomic Execution – means that **only one thread can be executing** a particular critical section. If any other process also wants to execute this critical section, it must wait until the first one finished



Conditions to be satisfied by a critical section implementation



- **Mutual Exclusion**- implies that out of all the threads only one can be implemented using various locking techniques.
- **Progress**: When the critical section is not being executed by any thread, the next thread (if any) should be allowed to access its critical section
- **Bounded Waiting**: Once a thread requests critical section access, there is a limit on the number of threads that access their critical sections before this thread's request is granted. After the limit is reached, this thread is allowed to execute its critical section code. This implies that there is no deadlock condition in which one thread hold the access to the critical section for infinite time.



Implementation of critical section

Entry Point – Locks all access to memory protected by critical section

{

Critical Section Code

}

Exit point – Releases the locks to the memory protected

{

Remainder Code

}

Example – a[ii] - critical section

For (ii = 0 ; ii < 6 ; ii++) {

$A[ii+1] = A[ii] + 1$

$A[ii] = A[ii-1] + 1$

}

ith iteration of loop $\rightarrow T_{i-1}$

T_4

$A[5] = A[4] + 1$

$A[4] = A[3] + 1$

T_5

$A[6] = A[5] + 1$

$A[5] = A[4] + 1$

T_6

$A[7] = A[6] + 1$

$A[6] = A[5] + 1$



Semaphores – critical section



- Implementation of critical section can be using a **semaphores**
- A thread should acquire a semaphore for entrance and should release it upon its exit
- Other threads are kept waiting until semaphore is released
- but can gain control of the cpu and execute the other code
- Once a thread comes out of semaphore, the waiting thread can acquire the semaphore, enter critical section and comes out
- Real example – one streaming per subscription. Here, the code used to stream the video is available only to one thread at a time



Data Race



- A data race occurs in a parallel program **when multiple threads access the same memory location** and at least one of those **accesses is a write**. It is a property of the execution of a program
- It is property of execution of a program. Thread analyzers can detect in multi-threaded program
- **Conditions** for data race
 - Multiple threads access the **same memory location**
 - At least one of these accesses is a **write**
 - There is no ordering of these **accesses through synchronization** such as locks



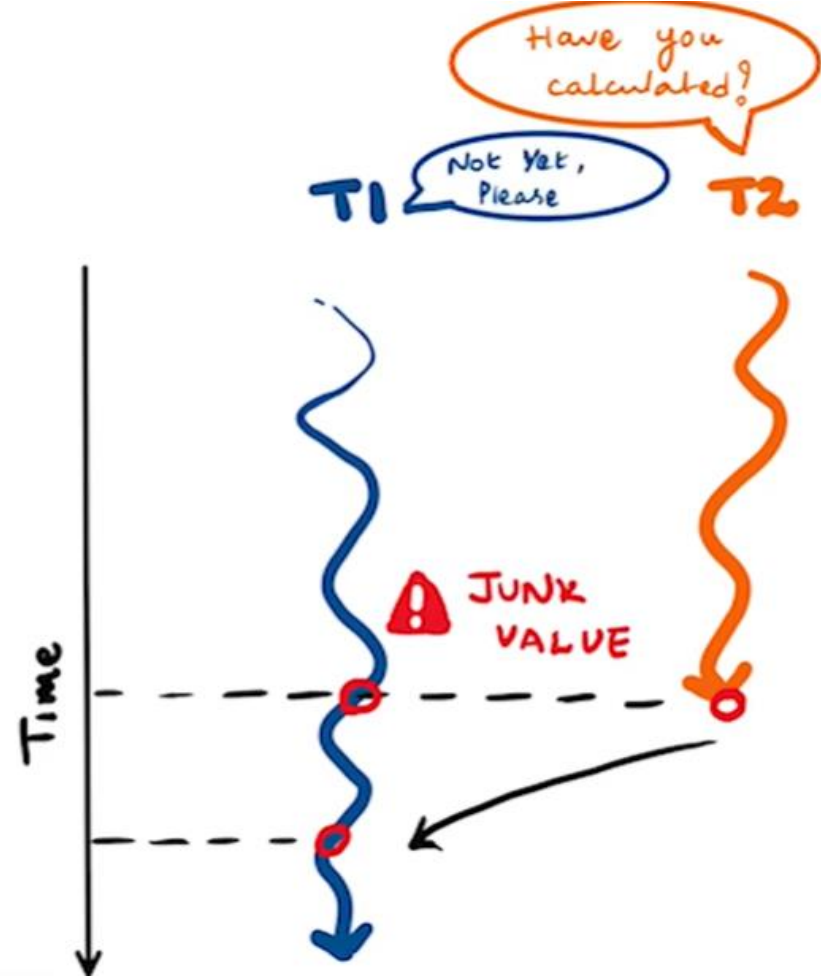
Why a data race occurs

- Bcoz Non deterministic order of accesses
- End results depend on the ordering of these accesses- **data dependency**
- Particularly, the write accesses because what is read by read accesses depends on the most recent write access.
- **Order of access can't be determined before execution**
- Bcoz the compiler can perform optimization and restructure the code
- Data race analogy – sufficient balance for outgoing transfer, receive money first then send it, relative order of transfer is imp
- Debugging- parallel computation becomes very difficult unless we have an idea of how far each of our threads have reached in the program
- Data races can be solved by using synchronization locks, barriers



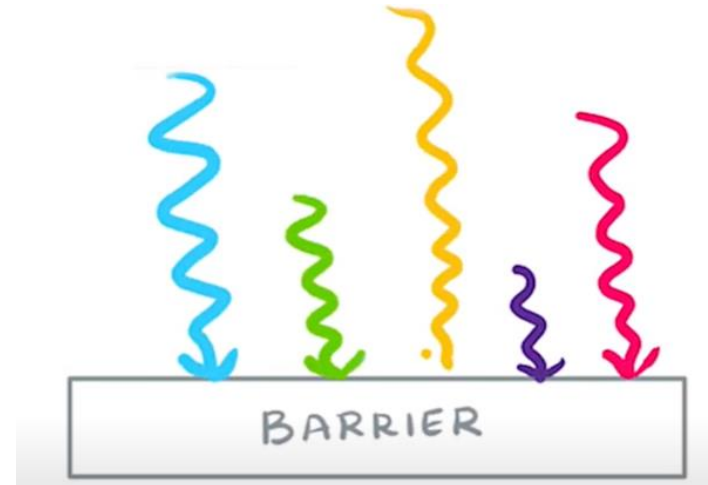
Synchronization

- Here, threads T1 takes longer time than T2 to execute
- Now, if T2 reaches at a point when it need to read a value computed by T1
 - If T1 has not finished execution then T2 ends up reading a junk value
 - T2 need to read the value only when T1 has written – guaranteed way
 - Can be fixed with synchronization
- Synchronization using barriers



Barriers

- A barrier is a point in parallel program where all points need to wait before any threads moves ahead
- A barrier is a point in parallel program where all points need to wait before any threads moves ahead
- Each thread which reaches barrier pauses its execution and waites for other to finish the execution before continuing its execution
- Eg Class of students writing down notes
- Or construction of a building





When can 2 statements execute in parallel

- When their order of execution must not matter
- In other words,

Statement1; statement2

- Must be equivalent to

Statement2; statement2

- Processors execute independently: no control over order of execution between processors



When can 2 statements execute in parallel

- Possibility 1

Processor1:

statement1;

Processor2:

statement2;

- Possibility 2

Processor1:

statement1;

Processor2:

statement2;

- Processors execute independently: no control over order of execution between processors



When can 2 statements execute in parallel



- . Their order of execution must not matter
- . In other words,

statement1; statement2;

must be equivalent to

statement2; statement1;



Example



a=1;

b=2;

- Statements can be executed in parallel.



Example



a=1;

b=a;

- Here, statements cannot be executed in parallel. However, program modifications may make it possible



Example



b = a;

a = 1;

- Statements can not be executed in parallel.



Example



a = 1;

a = 2;

- Statements can not be executed in parallel.



True dependence (RAW)



- For Statements S1, S2
S2 has a true dependence on S1
iff
S2 reads a value written by S1
- RAW – Read After Write



Anti-dependence(WAR)



- Statements S1, S2.

S2 has an anti-dependence on S1

iff

S2 writes a value read by S1.



Output Dependence(WAW)



- Statements S1, S2.

S2 has an output dependence on S1

iff

S2 writes a variable written by S1.



Types of Dependences



- . True (flow) dependence – RAW
- . Anti-dependence – WAR
- . Output dependence – WAW
- . So, S1 and S2 can execute in parallel iff there are no dependences between S1 and S2
 - . true dependences
 - . anti-dependences
 - . output dependences
- . Some dependences can be removed



Example



- Parallelism often occurs in loops.

```
for(i=0; i; i<100;i++)
```

```
    a[i] = i
```

- No dependences
- Iterations can be executed in parallel



Example



```
for(i=0; i<100; i++)  
{  
    a[i] = i;  
    b[i] = 2*i;  
}
```

- Iterations and statements can be executed in parallel.



Example



```
for(i=0;i<100;i++)
```

```
    a[i] = i;
```

```
for(i=0;i<100;i++)
```

```
    b[i] = 2*i;
```

- Iterations and loops can be executed in parallel.



Example



- . `for(i=0; i<100; i++)`
 `a[i] = a[i] + 100;`
- . There is a dependence ... on itself!
- . Loop is still parallelizable.



Example



```
for( i=0; i<100; i++ )  
    a[i] = f(a[i-1]);
```

- . Dependence between $a[i]$ and $a[i-1]$.
- . Loop iterations are not parallelizable.



Loop-Carried Dependence

- A loop-carried dependence is a dependence that is present only if the statements occur in two different instances of a loop
- Otherwise, we call it a loop-independent dependence
- Loop-carried dependences limit loop iteration parallelization



Example



```
for(i=0; i<100; i++ )
```

```
for(j=0; j<100; j++ )
```

```
    a[i][j] = f(a[i][j-1]);
```

- Loop-independent dependence on i.
- Loop-carried dependence on j.
- Outer loop can be parallelized, inner loop cannot.

Example

```
for( j=0; j<100; j++ )
```

```
for( i=0; i<100; i++ )
```

```
    a[i][j] = f(a[i][j-1]);
```

- Inner loop can be parallelized, outer loop cannot.
- Less desirable situation (finer-grain parallelism).
- Loop interchange is sometimes possible.



Example



$a = f(x);$

$b = g(x);$

- Statements could have a hidden dependence if f and g update the same variable.



Example



```
printf("a");  
printf("b");
```

- Statements have a hidden output dependence due to the output stream



Example



```
for(i=0; i<100; i++)
```

```
    a[i+10] = f(a[i]);
```

- Dependence between $a[10]$, $a[20]$, ...
- Dependence between $a[11]$, $a[21]$, ...
- Some parallel execution is possible.



Example



```
for( i=1; i<100;i++ ) {  
    a[i] = ...;  
    ... = a[i-1];  
}
```

- Dependence between $a[i]$ and $a[i-1]$
- Complete parallel execution impossible
- Pipelined parallel execution possible



Example



```
for( i=0; i<100; i++ )
```

```
    a[i] = f(a[indexa[i]]);
```

- Cannot tell for sure.
- Parallelization depends on user knowledge of values in `indexa[]`.
- User can tell, compiler cannot.



For Parallelization



- Statement order must not matter.
 - Statements must not have dependences.
 - Some dependences can be removed.
 - Some dependences may not be obvious.
-
- Parallelizing compilers analyze program dependences to decide parallelization.
 - In parallelization by hand, user does the same analysis.
 - Compiler more convenient and more correct
 - User more powerful, can use knowledge of data values



How to Write a Parallel Program



- . Identifying inherent parallelism
- . Partitioning or Decomposition
- . Dependency analysis
- . Communication
- . Programming Paradigms
- . Performance



Identification of Parallel region



- Identifying the parallelizable parts in a program is a crucial step in parallel programming.
- It involves analyzing the program's structure, algorithms, and data dependencies to identify portions that can be executed concurrently.
- Some techniques and approaches to identify parallelism are :
 - Manual Inspection
 - Static Code Analysis
 - Code Profiling



Identification (Contd.)

- Example:

```
int i, a[], b[], c[], x, t, y;
```

```
x=t-y;
```

```
for(i=0;i<100;i++)
```

```
    a[i]=b[i]*c[i];
```

```
y=t+x;
```

} Parallelizable



Decomposition

- Decomposition in parallel programming refers to the process of breaking down a problem or task into smaller, manageable parts that can be executed concurrently.
- Mainly there are 4 decomposition methods:
 - Data decomposition
 - Recursive decomposition
 - Exploratory decomposition
 - Speculative decomposition

Data Decomposition

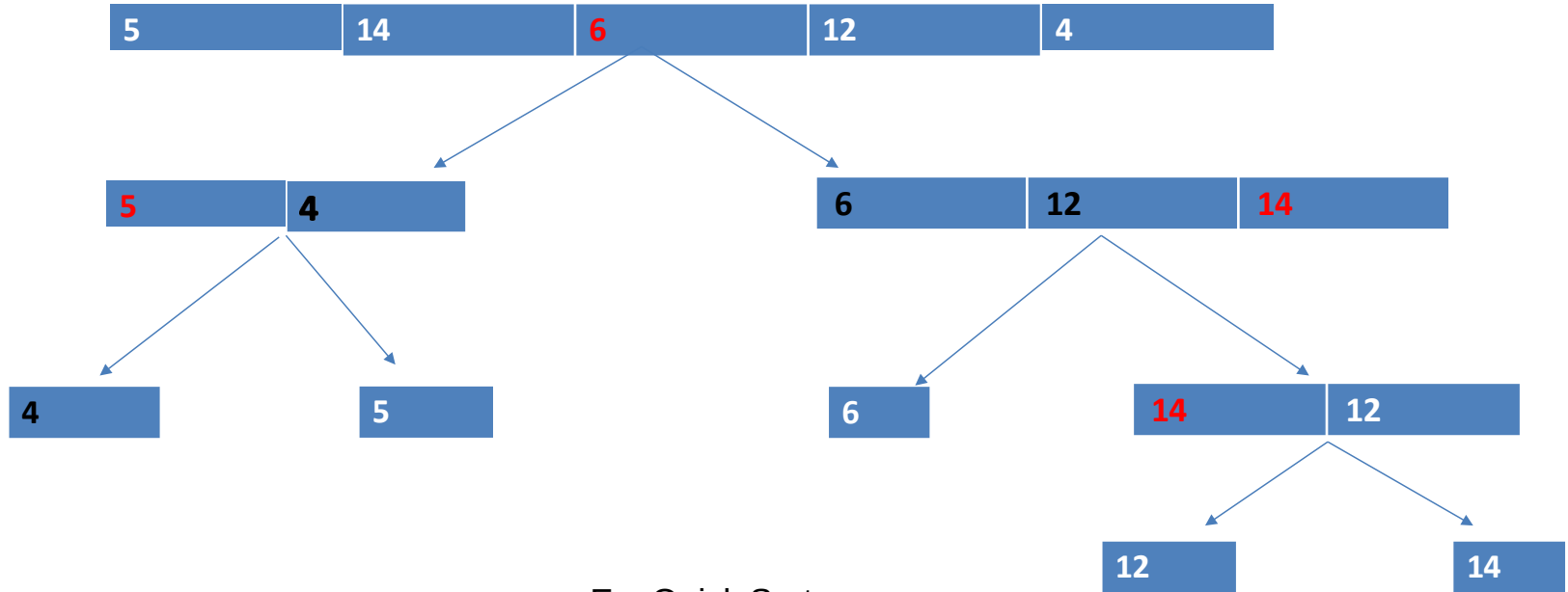
- Here the decomposition of computations is done in two steps:
 - Data partition and Task creation
- Example:

$$\begin{bmatrix} A1, & A1, \\ 1 & 2 \\ A2, & A2, \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} B1, & B1, \\ 1 & 2 \\ B2, & B2, \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} C1, & C1, \\ 1 & 2 \\ C2, & C2, \\ 1 & 2 \end{bmatrix}$$

Task 1: $C1.1 = A1.1+B1.1$ Task 2:
 $C1.2 = A1.2*B1.2$
Task 3: $C2.1 = A2.1*B2.1$ Task 4:
 $C2.2 = A2.2*B2.2$

Recursive decomposition

A problem is divided into smaller subproblems of the same type, and the same approach is recursively applied to each subproblem until a base case is reached. It is based on the principle of divide and conquer

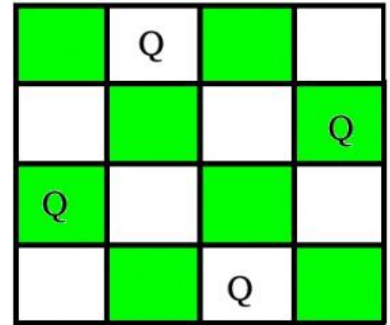


Eg. Quick Sort



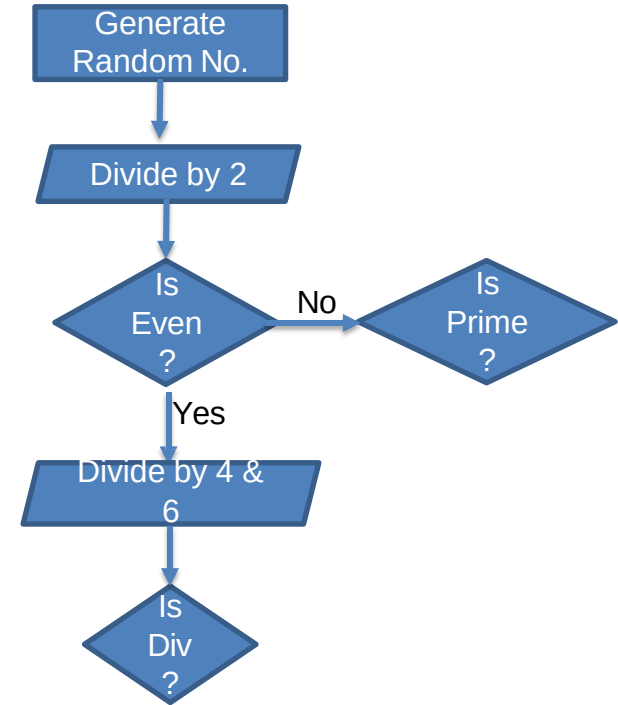
Exploratory Decomposition

- The idea behind exploratory decomposition is to dynamically explore and adapt the decomposition strategy based on insights gained during the execution of the parallel program. It allows for flexibility and adaptability in finding the most efficient decomposition of the problem.
- These problems typically involve the exploration (search) of a state space of solutions.
- Problems in this class include a variety of discrete optimization problems, theorem proving, game playing, etc.
- Example: n Queens problem



Speculative Decomposition

- This decomposition is used to exploit potential parallelism by speculatively executing multiple independent tasks or subproblems concurrently, without knowing for certain if all of them are necessary or will lead to a valid solution.
- While one task is performing the computation whose output is used in deciding the next configuration, other tasks can concurrently start the computations of the next stage.





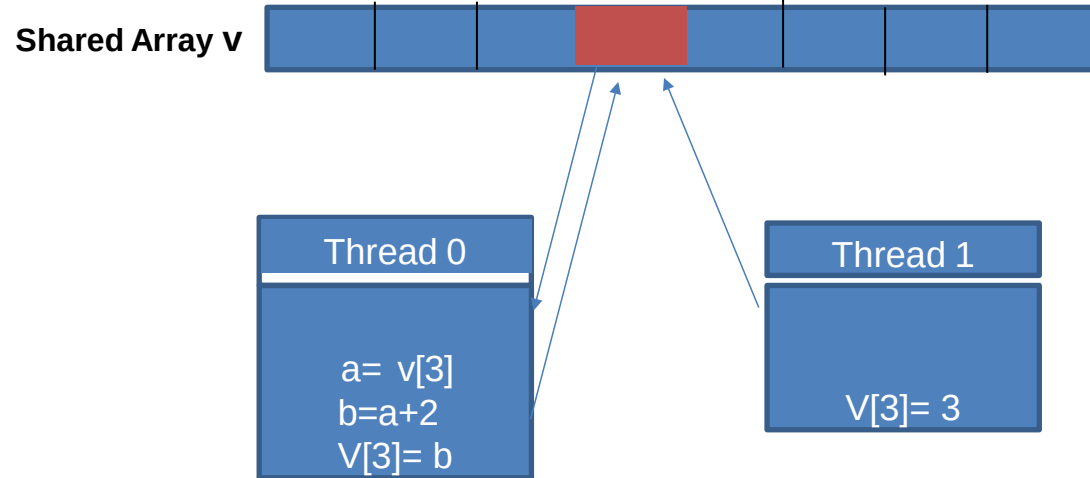
Data Dependency



- In parallel programs, order of statements in the code may not match the execution order;
 - data dependencies must be recognized and ensure that the computations are executed in the correct order.
- Data dependencies can be managed by
 - isolating dependencies within individual tasks or
 - coordinating the execution of multiple tasks.
- When data dependencies exist across parallel tasks, we do explicit coordination in execution of the task.

Data Dependency

This is an example of a *race condition* because two or more threads access the same location in memory and at least one of the threads writes to the shared location.



Even if the programmer intends these read and write operations to occur in a particular order, no steps were taken to manage the data dependency.

Data Dependency

Combined Thread ops
T1: $V[3] = 3$
T0: $a = v[3]$
T0: $b = a+2$
T0: $v[3] = b$

Result
$V[3] = 5$

Combined Thread ops
T0: $a = v[3]$
T1: $V[3] = 3$
T0: $b = a+2$
T0: $v[3] = b$

Result
$V[3] = 2$

Combined Thread ops
T0: $a = v[3]$
T0: $b = a+2$
T0: $v[3] = b$
T1: $V[3] = 3$

Result
$V[3] = 3$

- ✓ We might receive a different outcome every time the program runs if there is no mechanism.
- ✓ If the second thread (thread 1) writes a value into the vector and we want the first thread (thread 0) to use it, we need to use a technique to make sure thread 1 wrote the value before thread 0 read it.



Data Dependency

- Three types of data dependency or data hazards:
 - Read-after-Write (RAW)
 - Write-after-Read (WAR)
 - Write-after-Write (WAW)



RAW dependency

- Occurs when value produced by instruction is required by an subsequent instruction.
- Also known as true dependency or flow dependency
- Example:
 - ADD **R1**, R2, R3
 - SUB R4, **R1**, 5
- They are one of the most difficult dependencies to resolve in hardware.
(unparallelizable)



WAR dependency

- Occurs when an instruction writes to a location which has been read by a previous instruction
- Example:
 - `ADD R3, R2, R1`
 - `SUB R2, R5, 1`
- Instruction 2 must not produce its result in R2 before instruction 1 reads R2 , Otherwise instruction 1 would use the value produced by instruction 2 rather than the previous value of R2.

Also called name dependency, as they can be removed through renaming of register/variables

```
ADD R3, R2, R1
SUB R7, R5, 1
```



Output Dependency

- Occurs when a location is written by two instructions
- Example:
 - ADD **R3**, R2, R1
 - SUB R2, **R3**, 1
 - ADD R3, R2, R5
- Instruction 1 must produce its result in R3 before instruction 3 produces its result in R3 otherwise instruction 2 might use the wrong value of R3.
- Output Dependency are form of resource conflict (register reuse) and can be resolved by **register renaming**.



Loop carried Dependence



When a memory location is read or written in one iteration and written in another iteration.

- Loop carried dependencies – occurs between accesses across different loop iterations.
- Loop independent dependence - occurs between accesses in the same loop iteration

Example:

```
for (i=0; i<n; i++)  
{  
    A[i]=B[i];  
    A[i+1]=A[i]*2;  
}
```

Control flow dependency

- Execution of instructions depends on the outcome of a branch or control flow statement.
- Control dependencies restrict parallel execution because the outcome of one branch determines which instructions will be executed next

Example 1:

```
int x = 10; int y = 5; result;  
If (x > y) {  
    result = x - y; }  
else {  
    result = y - x; }
```

Example 2:

```
ADD R1, R2,  
                                     R  
BEQ Label3;  
SUB R4, R5, R9 ;  
ADD R7, R8, R9 ;  
                                     R  
Label: SUB6 R10, R11, R12 ;  
CMP R4, R1 ;
```



Communication



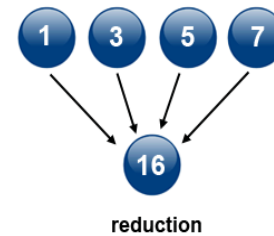
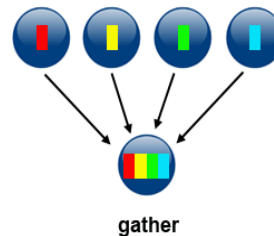
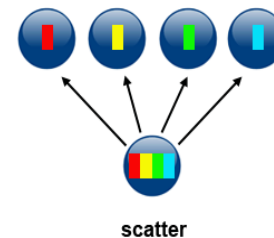
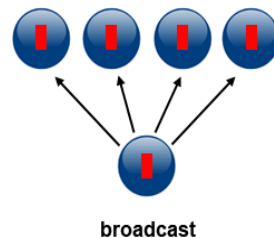
When do processors communicate ?

- To share data between different parallel tasks or processes
- To manage control flow dependencies,
E.g. when branching or making decisions based on results computed by other processors
- Communicating to share the error information to other processors for error handling or recovery
- Coordinate task dependencies, or assign subtasks to achieve efficient task coherence (Load Balance)

Communication

How do processors communicate ?

- In shared memory programming, use a semaphore or other locking mechanism for synchronization
- In distributed memory programming, use messages to communicate the results of earlier calculations.



Tips: Dependent calculations should be grouped into the same thread of execution if possible.



Synchronization



- ```
int main(...){
 int num_t, tid; //Shared
 #pragma omp parallel
 num_t = omp_get_num_threads()
 tid = omp_get_thread_num();
 printf("Hi from thread %d of %d \n", tid, numt);
 }
```
- What will this print?



# Synchronization (contd.)

## Normal scenario

Output 1:

Hi from thread 0 of 4  
Hi from thread 1 of 4  
Hi from thread 2 of 4  
Hi from thread 3 of 4

Output 2:

Hi from thread 2 of 4  
Hi from thread 1 of 4  
Hi from thread 0 of 4  
Hi from thread 3 of 4



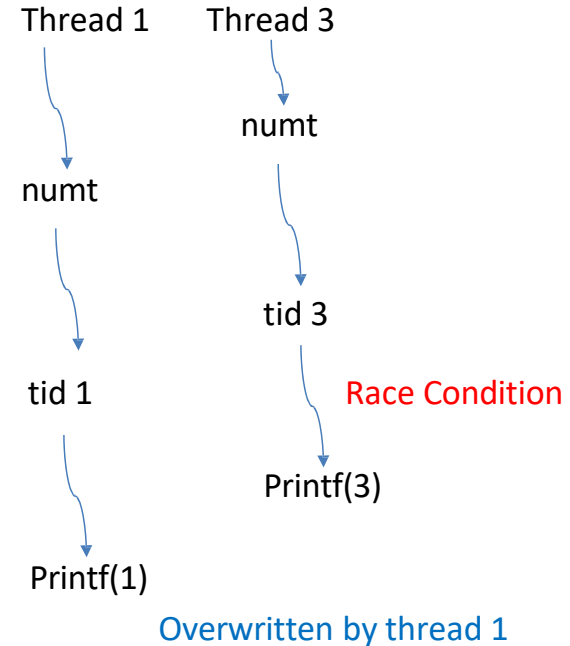
# Synchronization (contd.)



**If there is a race condition**

Output:

Hi from thread 2 of 4  
Hi from thread 1 of 4  
Hi from thread 0 of 4  
Hi from thread 1 of 4





# Synchronization (Contd.)



We need a mechanism to intelligently split the execution of a program

```
int main(...){
 int num_t, tid; //Shared
 #pragma omp parallel private(tid) shared(numt)
 num_t = omp_get_num_threads()
 tid = omp_get_thread_num();
 printf("Hi from thread %d of %d \n", tid, numt);
}
```

- Synchronization overhead
- In case barrier is used, the slowest task determines the speed of the whole calculation.



# Granularity

- **Granularity** (or grain size) of a task is a measure of the amount of work (or computation) which is performed by that task.
- Or  $\text{Granularity} = \text{computation time} / \text{communication time}$
- Two types of Granularity:
  - Fine-grained granularity :
    - Smaller 'chunk' size.
    - Relatively less computation work between communication events.
    - Better with shared memory model.
    - Less communication overhead.
    - Better choice while handling load balancing.

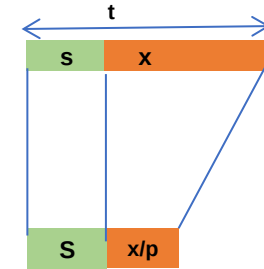
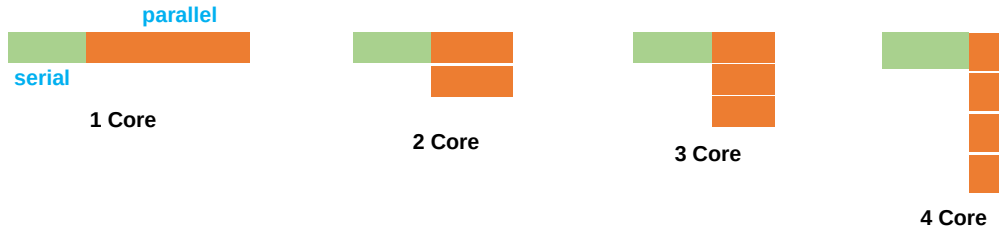


# Granularity(Contd.)



- Coarse-grained granularity.
  - Bigger 'chunk size'.
  - Computation to communication ratio is high.
  - Tasks completes more work with relatively less communication/synchronization events.
  - Well suited for distributed memory system.
  - Used in most cases for improved performance because of less communication requirement.

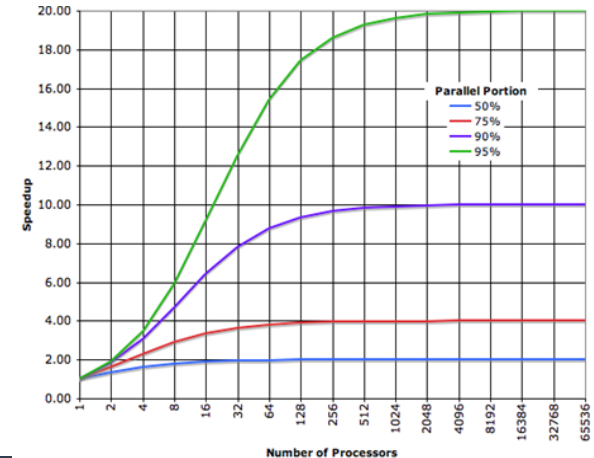
# Amdahl's Law



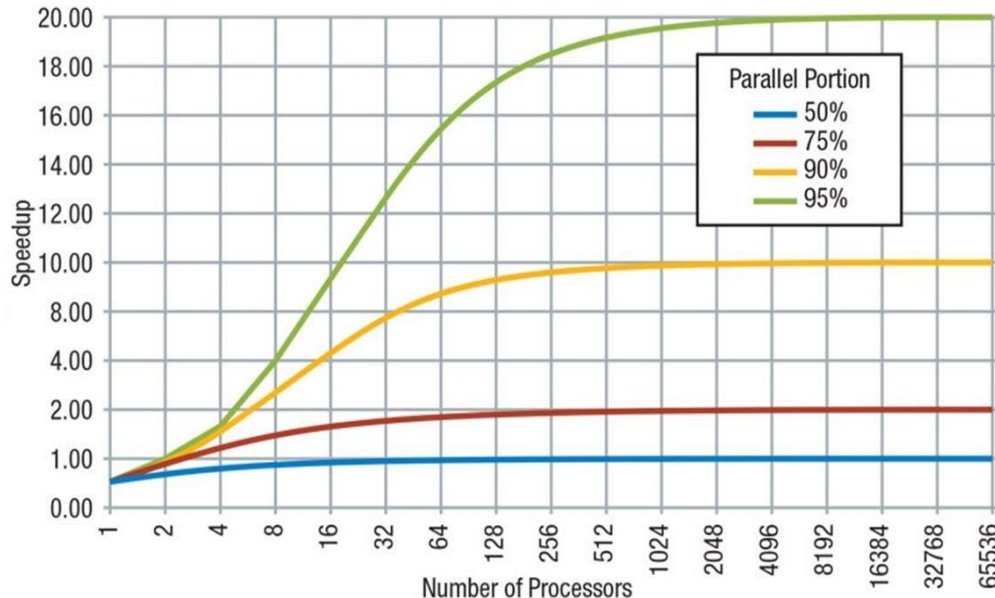
According to Amdahl's Law, the speedup of a program is limited by the fraction of the program that cannot be parallelized and applications can almost never be completely parallelized.

•  $S_p$  is the speedup

$$S_p = \frac{1}{(1-X)+(X/P)}$$



# Amdahl's law



According to Amdahl's Law, the speedup of a program is limited by the fraction of the program that cannot be parallelized and applications can almost never be completely parallelized.

$$S(p) = \frac{1}{(1-P) + \frac{P}{N}}$$

Even if you have an infinite number of processors(n), there will always be a limit to the achievable speedup based on the serial portion of the program (1 - p).

## Matrix-Matrix Addition

|    |           |           |           |           |           |           |           |           |
|----|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| P0 | $A_{0,0}$ | $A_{0,1}$ | $A_{0,2}$ | $A_{0,3}$ | $B_{0,0}$ | $B_{0,1}$ | $B_{0,2}$ | $B_{0,3}$ |
| P1 | $A_{1,0}$ | $A_{1,1}$ | $A_{1,2}$ | $A_{1,3}$ | $B_{1,0}$ | $B_{1,1}$ | $B_{1,2}$ | $B_{1,3}$ |
| P2 | $A_{2,0}$ | $A_{2,1}$ | $A_{2,2}$ | $A_{2,3}$ | $B_{2,0}$ | $B_{2,1}$ | $B_{2,2}$ | $B_{2,3}$ |
| P3 | $A_{3,0}$ | $A_{3,1}$ | $A_{3,2}$ | $A_{3,3}$ | $B_{3,0}$ | $B_{3,1}$ | $B_{3,2}$ | $B_{3,3}$ |

# Approach For Parallelizing Matrix Matrix Addition

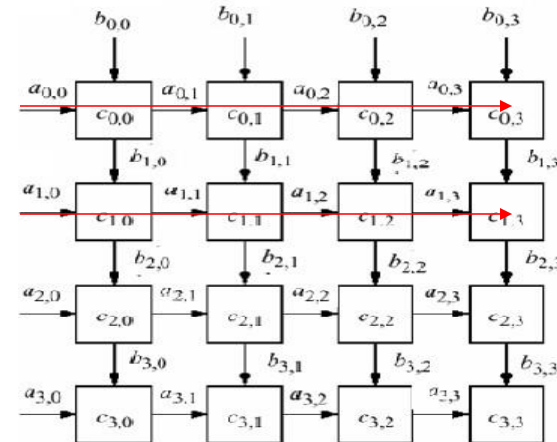
|           |           |           |           |
|-----------|-----------|-----------|-----------|
| $A_{0,0}$ | $A_{0,1}$ | $A_{0,2}$ | $A_{0,3}$ |
| $A_{1,0}$ | $A_{1,1}$ | $A_{1,2}$ | $A_{1,3}$ |
| $A_{2,0}$ | $A_{2,1}$ | $A_{2,2}$ | $A_{2,3}$ |
| $A_{3,0}$ | $A_{3,1}$ | $A_{3,2}$ | $A_{3,3}$ |

MATRIX A

|           |           |           |           |
|-----------|-----------|-----------|-----------|
| $B_{0,0}$ | $B_{0,1}$ | $B_{0,2}$ | $B_{0,3}$ |
| $B_{1,0}$ | $B_{1,1}$ | $B_{1,2}$ | $B_{1,3}$ |
| $B_{2,0}$ | $B_{2,1}$ | $B_{2,2}$ | $B_{2,3}$ |
| $B_{3,0}$ | $B_{3,1}$ | $B_{3,2}$ | $B_{3,3}$ |

MATRIX B

Resultant  
Matrix



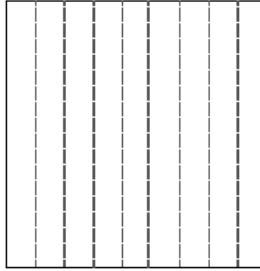
Matrix A and Matrix B are divided among 4 Processors P0, P1, P2, P3

# Partitioning & Mapping

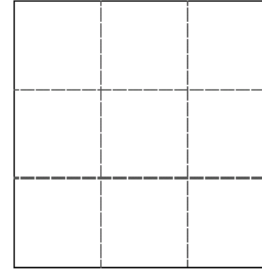
## Regular Partitioning



By Row



By Column



By Block

**Different parallel formulations :-**

**row-wise striping, column-wise striping, or checkerboard striping.**



# Parallel Programming Paradigms

- On a single machine with shared memory
  - **Pthreads**
  - **OpenMP 3**
- On machines connected via network and have no shared memory
  - **MPI**
  - **PGAS**
- Accelerators, offload tasks from CPU to it
  - **CUDA**
  - **OpenACC /OpenMP 4**
  - **OpenCL**